Quickstarts

Deploy contracts

Copy page

Deploy pre-audited smart contract templates on Arc with Circle Contracts.

This tutorial guides you through deploying smart contracts on Arc Testnet with **Circle Contracts**. You'll create a **Circle Dev-Controlled SCA Wallet**, then deploy pre-audited contract templates (ERC-20, ERC-721, ERC-1155, Airdrop). With SCA wallets, **Circle Gas Station** automatically sponsors your transaction fees on Arc Testnet.

These pre-audited templates represent building blocks: ERC-20 for money and liquidity, ERC-721 for identity and unique rights, ERC-1155 for scalable financial instruments, and Airdrops for distributing incentives. To learn more about available templates, visit the **Templates Overview** to review all templates and their options.

Prerequisites

To complete this tutorial, you need:

1. **Node.js v22+** installed
2. **Circle Developer Account** - **Sign up** on the Developer Console
3. **API Key** - Create in the Console: **Keys** → **Create a key** → **API key** → **Standard Key**
4. **Entity Secret** - Required to initialize the Circle Dev-Controlled Wallets SDK. Learn how to **register your Entity Secret**

Step 1. Set up your project

Before deploying any template, you need a working project and a funded dev-controlled wallet on Arc Testnet. Complete the steps in this section once. Then reuse the same wallet and credentials across all template deployments below.

1.1. Create the project and install dependencies

Create a new directory. Navigate to it and start a new project with default settings.

Node.js Python

```
mkdir hello-arc
cd hello-arc
npm init -y
npm pkg set type=module

# Add run scripts for wallet creation and contract deployment
npm pkg set scripts.create-wallet="tsx --env-file=.env create-wallet.ts"
npm pkg set scripts.deploy-erc20="tsx --env-file=.env deploy-erc20.ts"
npm pkg set scripts.deploy-erc721="tsx --env-file=.env deploy-erc721.ts"
npm pkg set scripts.deploy-erc1155="tsx --env-file=.env deploy-erc1155.ts"
npm pkg set scripts.deploy-airdrop="tsx --env-file=.env deploy-airdrop.ts"
```

In the project directory, install the **Circle Dev-Controlled Wallets SDK** and the **Circle Contracts SDK**. Dev-Controlled Wallets are Circle-managed wallets that your app controls via APIs. You can deploy contracts and submit transactions without managing private keys directly. You can also call the **Circle Wallets API** and **Circle Contracts API** directly if you can't use the SDKs in your project.

Node.js Python

```
npm install @circle-fin/developer-controlled-wallets @circle-fin/smart-contrac
npm install --save-dev tsx typescript @types/node
```

1.2. Configure TypeScript (optional)

Create a `tsconfig.json` file:

Node.js

```
npx tsc --init
```

Then, edit the `tsconfig.json` file:

Node.js

```
cat <<'EOF' > tsconfig.json
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "types": ["node"]
  }
}
EOF
```

1.3. Set environment variables

Create a `.env` file in the project directory with your Circle credentials. Replace these placeholders with your own credentials:

.env

```
CIRCLE_API_KEY=YOUR_API_KEY
CIRCLE_ENTITY_SECRET=YOUR_ENTITY_SECRET
CIRCLE_WEB3_API_KEY=YOUR_API_KEY
```

- `CIRCLE_API_KEY` is your Circle Developer API key for Wallets and Contracts API requests.
- `CIRCLE_ENTITY_SECRET` is your registered entity secret used to authorize developer-controlled wallet operations.
- `CIRCLE_WEB3_API_KEY` is the Python SDK compatibility variable and should use the same value as `CIRCLE_API_KEY`.

The npm run commands in this tutorial load variables from `.env` using Node.js native env-file support.

Prefer editing `.env` files in your IDE or editor so credentials are not leaked to your shell history.

This tutorial adds runtime values such as wallet IDs, transaction IDs, and contract IDs later in the flow. Keep those derived values aligned with the script outputs as you progress through the deployment steps.

Step 2. Set up your wallet

In this step, you create a dev-controlled wallet and fund it for contract deployment on Arc Testnet. If you already have a funded Arc Testnet dev-controlled wallet, skip to **the contract templates section**.

2.1. Create a wallet

Import the Wallets SDK and start the client with your API key and Entity Secret. Dev-controlled wallets are created in a **wallet set**. The wallet set is the source from which wallet keys are derived.

create-wallet.ts create_wallet.py

```
import { initiateDeveloperControlledWalletsClient } from "@circle-fin/developer

const client = initiateDeveloperControlledWalletsClient({
  apiKey: process.env.CIRCLE_API_KEY,
  entitySecret: process.env.CIRCLE_ENTITY_SECRET,
});

// Create a wallet set
const walletSetResponse = await client.createWalletSet({
  name: "Wallet Set 1",
});

// Create a wallet on Arc Testnet
const walletsResponse = await client.createWallets({
  blockchains: ["ARC-TESTNET"],
  count: 1,
  walletSetId: walletSetResponse.data?.walletSet?.id ?? "",
  accountType: "SCA",
});

console.log(JSON.stringify(walletsResponse.data, null, 2));
```

Run the script:

Node.js Python

```
npm run create-wallet
```

Response:

> If you're calling the API directly, you'll need two requests. One to **create the wallet set**. One to **create the wallet**.
>
> Be sure to replace the **Entity Secret ciphertext** and the idempotency key in your request. If you're using the SDKs, this is handled for you.

You should now have a newly created dev-controlled wallet. The API response will look similar to the following:

```
{
  "wallets": [
    {
      "id": "45692c3e-2ffa-5c5b-a99c-61366939114c",
      "state": "LIVE",
      "walletSetId": "ee58db40-22b4-55cb-9ce6-3444cb6efd2f",
      "custodyType": "DEVELOPER",
      "address": "0xbcf83d3b112cbf43b19904e376dd8dee01fe2758",
      "blockchain": "ARC-TESTNET",
      "accountType": "SCA",
      "updateDate": "2026-01-20T09:39:16Z",
      "createDate": "2026-01-20T09:39:16Z",
      "scaCore": "circle_6900_singleowner_v3"
    }
  ]
}
```

> **Why SCA wallets?** Smart Contract Accounts (SCA) on Arc Testnet work with **Gas Station** to automatically sponsor transaction fees. Learn more about **Gas Station policies and setup**.

ERC-20 ERC-721 ERC-1155 Airdrop

Deploy an ERC-20 contract

ERC-20 is the standard for fungible tokens. Use this template for tokenized assets, treasury instruments, governance tokens, or programmable money.

Step 3: Prepare for deployment

3.1. Get your wallet information

Retrieve your wallet ID from Step 2. Ensure:

- Wallet custody type is **Dev-Controlled**
- Blockchain is **Arc Testnet**
- Account type is **SCA** (Smart Contract Account, recommended for Gas Station compatibility)

Note your wallet's address for subsequent steps.

3.2. Understand deployment parameters

Parameter	Description
`idempotencyKey`	A unique value to prevent duplicate requests.
`name`	The offchain contract name (visible in Circle Console only). Use `MyTokenContract` .
`walletId`	The ID of the wallet deploying the contract. Use your dev-controlled wallet ID.
`templateId`	The template identifier. Use `a1b74add-23e0-4712-88d1-6b3009e85a86` for ERC-20. See **Templates**.
`blockchain`	The network to deploy onto. Use `ARC-TESTNET` .
`entitySecretCiphertext`	The re-encrypted entity secret. See **Entity Secret Management**.
`feeLevel`	The fee level for transaction processing. Use `MEDIUM` .

Parameter	Description
`templateParameters`	The onchain initialization parameters (see below).

›

3.3. Template parameters

Required Parameters:

Parameter	Type	Description
`name`	String	The onchain contract name. Use `MyToken`.
`defaultAdmin`	String	The address with administrator permissions. Use your Dev-Controlled Wallet address.
`primarySaleRecipient`	String	The address that receives proceeds from first-time sales. Use your wallet address.

Optional Parameters:

Parameter	Type	Description
`symbol`	String	The token symbol (for example, `MTK`).
`platformFeeRecipient`	String	The address that receives platform fees from sales. Set this when implementing platform fee revenue share.
`platformFeePercent`	Float	The platform fee percentage as decimal (for example, `0.1` for 10%). Requires `platformFeeRecipient`.
`contractUri`	String	The URL for the contract metadata.
`trustedForwarders`	Strings[]	A list of addresses that can forward ERC2771 meta-transactions to this contract.

Step 4: Deploy the smart contract

Deploy by making a request to `POST /templates/{id}/deploy`:

deploy-erc20.ts deploy_erc20.py cURL

```
import { initiateSmartContractPlatformClient } from "@circle-fin/smart-contrac
                                                  ›
const circleContractSdk = initiateSmartContractPlatformClient({
  apiKey: process.env.CIRCLE_API_KEY,
  entitySecret: process.env.CIRCLE_ENTITY_SECRET,
});

const response = await circleContractSdk.deployContractTemplate({
  id: "a1b74add-23e0-4712-88d1-6b3009e85a86",
  blockchain: "ARC-TESTNET",
  name: "MyTokenContract",
  walletId: process.env.WALLET_ID,
  templateParameters: {
    name: "MyToken",
    symbol: "MTK",
    defaultAdmin: process.env.WALLET_ADDRESS,
    primarySaleRecipient: process.env.WALLET_ADDRESS,
  },
  fee: {
    type: "level",
    config: {
      feeLevel: "MEDIUM",
    },
  },
});

console.log(JSON.stringify(response.data, null, 2));
```

Run the script:

Node.js Python

```
npm run deploy-erc20
```

Response:

```
{
  "contractIds": ["019c053d-1ed1-772b-91a8-6970003dad8d"],
  "transactionId": "5b6185b2-f9a1-5645-9db2-ca5d9a330794"
}
```

A successful response indicates deployment has been **initiated**, not completed. Use the `transactionId` to check the deployment status in the next step.

4.1. Check deployment status

You can check the status of the deployment from the Circle Developer Console or by calling `GET /transactions/{id}`.

After running the deployment script, copy the `transactionId` from the response and update your `.env` file with `TRANSACTION_ID={your-transaction-id}`. Then run the check-transaction script to verify deployment status.

check-transaction.ts check_transaction.py

```
import { initiateDeveloperControlledWalletsClient } from "@circle-fin/develope

const circleDeveloperSdk = initiateDeveloperControlledWalletsClient({
  apiKey: process.env.CIRCLE_API_KEY,
  entitySecret: process.env.CIRCLE_ENTITY_SECRET,
});

const transactionResponse = await circleDeveloperSdk.getTransaction({
  id: process.env.TRANSACTION_ID!,
});

console.log(JSON.stringify(transactionResponse.data, null, 2));
```

Run the script:

Node.js Python

```
npm pkg set scripts.check-transaction="tsx --env-file=.env check-transaction.t
npm run check-transaction
```

Transaction status may show PENDING immediately after deployment. Wait 10-30 seconds and re-run check-transaction to see COMPLETE status.

Response:

```
{
  "transaction": {
    "id": "601a0815-f749-41d8-b193-22cadd2a8977",
    "blockchain": "ARC-TESTNET",
    "walletId": "45692c3e-2ffa-5c5b-a99c-61366939114c",
    "sourceAddress": "0xbcf83d3b112cbf43b19904e376dd8dee01fe2758",
    "contractAddress": "0x281156899e5bd6fecf1c0831ee24894eeeaea2f8",
    "transactionType": "OUTBOUND",
    "custodyType": "DEVELOPER",
    "state": "COMPLETE",
    "amounts": [],
    "nfts": null,
    "txHash": "0x3bfbab5d5ce0d1a5d682cbc742d3940cf59db0369d173b71ba2a3b8f43bfbc
    "blockHash": "0x7d12148f9331556b31f84f58a41b7ff16eaaa47940f9e86733037d7ab74
    "blockHeight": 23686153,
    "userOpHash": "0x66befac1a371fcdddf1566215e4677127e111dff9253f306f7096fed86
    "networkFee": "0.044628774800664",
    "firstConfirmDate": "2026-01-26T08:59:56Z",
    "operation": "CONTRACT_EXECUTION",
    "feeLevel": "MEDIUM",
    "estimatedFee": {
      "gasLimit": "500797",
      "networkFee": "0.16506442157883425",
      "baseFee": "160",
      "priorityFee": "9.60345525",
      "maxFee": "329.60345525"
    },
    "refId": "",
    "abiFunctionSignature": "mintTo(address,uint256)",
    "abiParameters": [
      "0xbcf83d3b112cbf43b19904e376dd8dee01fe2758",
      "1000000000000000000"
    ],
    "createDate": "2026-01-26T08:59:54Z",
    "updateDate": "2026-01-26T08:59:56Z"
  }
}
```


4.2. Get the contract address

After deployment completes, retrieve the contract address using `GET /contracts/{id}`.

After deployment completes, copy the `contractIds[0]` from the deployment response and update your `.env` file with `CONTRACT_ID={your-contract-id}`. Then run the get-contract script to retrieve the contract address.

get-contract.ts get_contract.py

```
import { initiateSmartContractPlatformClient } from "@circle-fin/smart-contrac

const circleContractSdk = initiateSmartContractPlatformClient({
  apiKey: process.env.CIRCLE_API_KEY,
  entitySecret: process.env.CIRCLE_ENTITY_SECRET,
});

const contractResponse = await circleContractSdk.getContract({
  id: process.env.CONTRACT_ID!,
});

console.log(JSON.stringify(contractResponse.data, null, 2));
```

Run the script:

Node.js Python

```
npm pkg set scripts.get-contract="tsx --env-file=.env get-contract.ts"
npm run get-contract
```

Response:

```
{
  "contract": {
    "id": "b7c35372-ce69-4ccd-bfaa-504c14634f0d",
    "contractAddress": "0x1234567890abcdef1234567890abcdef12345678",
    "blockchain": "ARC-TESTNET",
    "status": "COMPLETE"
  }
}
```

Once your contract is deployed, you can interact with it from your application. You'll be able to view the contract both in the Circle Developer Console and on the Arc Testnet Explorer.

> **Initial Supply:** The contract starts with 0 token supply at deployment. Use the `mintTo` function to create tokens and assign them to addresses as needed.

Summary

After completing this tutorial, you've successfully:

- Created a dev-controlled wallet on Arc Testnet
- Funded your wallet with testnet USDC
- Deployed a smart contract using Contract Templates
- Retrieved your contract address

Was this page helpful? Yes No

llms.txt Bridge USDC to Arc

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.